

Five Primitives Are All You Need: Building an Agent Harness

Chase Dovey
Average Joes Lab

October 2026 Preprint

Abstract

People describe LLM agents with words that don't line up: framework, scaffold, runtime, orchestration layer, harness. Recent work argues that this outer layer drives much of how an agent performs, and that comparing agents without it misleads. But nobody has said what the layer is made of. I give an answer.

An agent is two things. **The model** is a function, $TokensOut = Model(TokensIn)$. **The harness** is everything else. So **Agent = Harness(Model)**. Every harness, I argue, is made of **five mechanistic primitives**:

- **control flow** decides what runs, in what order, and when to stop, and it wraps the other four;
- **input** gathers what goes to the model, from a person or the world;
- **context** decides what the request holds and how it fits;
- **the model interface** sends the request and gets the response back;
- **output** handles the response, by showing it to a person or running it as a tool.

The method is one question, asked again and again: *what is it, and by what mechanism does it work?* Each primitive is defined as much by what it never does as by what it does.

The core of the paper is a build. I start from a single API call and add one primitive at a time, and each step's missing piece turns out to be exactly the next primitive. The finished agent, quark, is 48 lines of Python, with nothing outside the five.

Two results follow.

1. **Production concerns fold back into the primitives.** Observability, guardrails, sandboxing, resilience, performance and evaluation look like new building blocks. Each is hardening inside the primitives it touches.
2. **The vocabulary holds on real systems.** I take apart three production coding agents (Claude Code, OpenAI Codex CLI and opencode) part by part. Nothing in them needs a sixth primitive. The study also found where my original rules were too loose, and I tighten them here.

A case study, in which quark wrote much of the material that teaches it, shows the vocabulary used for diagnosis.

Companion artifact (an open course with runnable code): <https://github.com/averagejoeslab/building-agents> [1].

1 Introduction

I’ve watched a lot of talks about agents. Speaker after speaker uses a different word for the same thing: the *framework*, the *scaffold*, the *runtime*, the *orchestration layer*, the *harness*. But when they show how their system works, it’s the same mechanism every time.

- A model is called.
- Something decides what goes into the call.
- Something acts on what comes back.
- Something decides whether to go again.

The words don’t line up. The mechanisms do.

That gap costs something:

- **Diagnosis.** If you can’t name the parts of your agent, you can’t say which part failed.
- **Comparison.** You can’t compare two systems whose authors describe them differently.
- **The market.** The field keeps meeting the same few ideas as new products: memory layers, gateways, tool protocols, tracing platforms, sandboxes, evaluation suites. Without a map, it’s hard to tell whether each is new or an old idea renamed.

The outer layer now has a name, *harness engineering*, and evidence that it matters:

- Zhang et al. [2] argue that, among comparable frontier models, the harness can drive more of the variance in performance than the choice of model, so agents shouldn’t be compared without disclosing their harness.
- Fan et al. [3] hold a coding harness’s loop fixed and vary three of its parts: planning, the action space and context management. Each one changes how the agent behaves.

Both treat the harness as what matters. Fan et al. pick parts of a harness to vary. I ask what the complete set of parts is.

My answer is **reductive**: take an agent apart until the parts stop being shared across systems. It is also **mechanistic**: define each part by what it does on the path from a request to a response, not by what it’s called or who sells it.

Contributions. The first three are the paper. The rest follow from them.

1. **Agent = Harness(Model):** the model is a fixed function, and everything around it is harness (§2).
2. **Five primitives of a harness**, each defined by what it does and what it never does, with rules for the hard cases at their boundaries (§3–4).
3. **A build showing the five are enough:** a working agent, built one primitive at a time, where each step’s missing piece is the next primitive (§5).
4. **Production hardening folds back** into the primitives instead of adding new ones (§6).
5. **A decomposition of three production harnesses**, Claude Code, Codex and opencode. Nothing needs a sixth primitive, and the study sharpens the rules (§7).
6. **A case study** in which the agent built here wrote much of the material that teaches it (§8).

The paper comes with a companion artifact, *harness-engineering* [1]: an open course with runnable code. It builds the harness of §5 and the production layers of §6 one lesson at a time, and keeps the full transcripts of every run. The paper stands on its own, though. Every listing and run it relies on is reproduced in the appendices (A–D), and the artifact has them in full.

2 Two primitives: the model and the harness

Start with an agent and ask what it's made of. Two things.

The model predicts. Given tokens, it produces the tokens most likely to come next:

TokensOut = Model(TokensIn)

From the outside, the model is a function. Between calls it has no memory. It can't read a terminal or run a command. It can't decide to be called again. All it knows about this moment is in the tokens it's handed. Building it is *model development*: training data, compute, and the methods to train and evaluate a model. That's its own discipline, and I treat what it produces as fixed.

The harness does everything else. It decides:

- how inputs are gathered;
- how they're presented to the model;
- how the model is called;
- how its outputs are handled;
- how information flows between all of them.

Agent = Harness(Model)

The model goes inside, and the harness wraps it. Harness engineering is building that second part.

The split is blunt on purpose. Anything that isn't the model's weights is harness: a system prompt, a tool, a retry policy, a memory store. Two agents on the same model differ only in their harnesses, so a comparison that leaves out the harness is comparing the wrong thing.

2.1 Method

The method is one question:

What is it, and by what mechanistic primitives does it work?

Asking it gives a sequence of answers:

1. **Ask it of an agent** and you get a model and a harness.
2. **Ask it of a harness** and you get five primitives (§3).
3. **Ask it once more**, of a primitive, and the answers stop being shared. One harness's input reads a terminal, another's reads a Slack channel, and a third's reads a webhook.

That's where taking apart ends. A primitive is the deepest level at which every harness still has the same parts.

Each primitive gets two definitions:

- **positive:** what it does;
- **negative:** what it never does, naming which other primitive owns that job.

The negative definitions do most of the work. They make the primitives disjoint, so every piece of harness code has exactly one home. A piece is placed by **what it does, not where the code sits** or what it's called (§4).

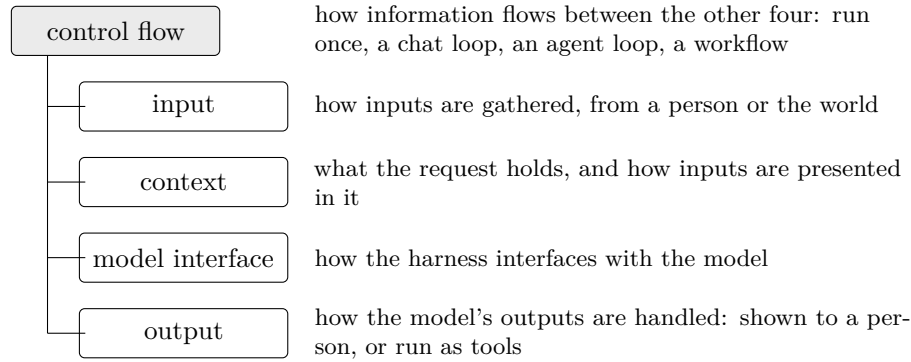


Figure 1: The five primitives of a harness. Control flow wraps the other four.

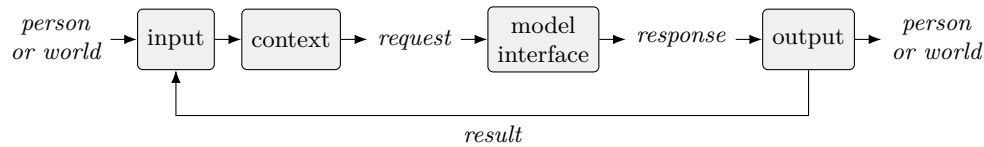


Figure 2: One pass through a harness. A tool's result comes back as input; control flow decides whether to go around again.

2.2 Scope

The primitives describe what happens on the path of a request at run time. Some of what ships with a harness never runs on that path, and I leave it out of scope the way I leave out model development:

- **Transport and hosting:** a client/server split, a remote workspace, the process that serves the UI.
- **Persistence:** the database.
- **Event wiring:** an internal event bus.
- **Configuration and packaging:** loading settings, installing plugins.

They carry, store or configure the primitives without being a step in a pass. Where they do something on the path, such as a saved transcript the harness replays, that piece is placed like any other.

3 The five primitives

One pass through a harness follows one path:

1. Input gathers what goes in.
2. Context assembles it into a request and makes it fit.
3. The model interface sends the request and gets the response.
4. Output handles the response, by showing it to a person or running a tool. A tool's result comes back as input.
5. Control flow decides what happens at the end: go around again, give a person a turn, or stop.

3.1 Model interface

Does: sends tokens to the model as a request and gets tokens back as a response.

Never does: decide when to call (control flow), gather what goes in (input), decide how it's presented (context), or act on what comes back (output).

It's the thinnest primitive, because of where it sits. The other four live on the harness's side. The model interface is the boundary: in *Agent = Harness(Model)*, it's the parentheses. At its smallest it's one SDK call. Bigger ones vary along a few lines:

- **Where the request goes:** a hosted API, a local model, a gateway.
- **How it travels:** an SDK, raw HTTP, a layer that translates between providers.
- **Which model answers:** one model, or a backup for when the first is down.
- **How the response arrives:** all at once, or streamed.
- **How many requests go:** one at a time, or a batch.
- **What happens when a request fails:** retry, wait, time out.
- **The request's settings:** output cap, thinking effort, whether you get a summary of the thinking.

All of it is about getting one request there and one response back.

3.2 Input

Does: gathers what goes to the model, from a **person** (a task, a question, a message) or from **the world** (what happened when a tool ran: what it printed, its exit code, whether it failed).

Never does: decide how what it gathered is laid out in the request (context), send it (model interface), or decide whether to go again (control flow).

Input is the harness's afferent pathway, carrying signals in.

- **Where it comes from:** command-line arguments, a prompt, a pipe, a chat app, a webhook, a schedule.
- **What it can be:** text, images, files.

Wherever it comes from, input does one thing: it brings something from outside the harness to the edge of the request.

3.3 Output

Does: handles the model's response, by **showing it** to a person or **running it** as a tool.

Never does: decide whether a tool's result goes back to the model (control flow), or what the next request holds (context).

Output is the efferent pathway, carrying actions out. The model can't act, but it can ask. Its response can include a request to use a tool: a name and arguments, in a shape the harness described to it. Output runs the request, and what happened comes back as input. **The model asks; the harness acts.** Output's choices include:

- **Which tools exist:** one general tool, or many specific ones.
- **Where they run:** this machine, a container, a remote host, the provider.
- **How they run:** one after another, or at the same time.
- **When not to run:** a request cut off mid-generation is incomplete, and running half a command is worse than running none.

- **What happens when a tool fails or hangs.**

Input and output are built separately, but they're two ends of one exchange, and they meet at the tool. A tool needs both: output to run it, and input to bring back what happened.

3.4 Control flow

Does: **sequences** the other four, passing what one produced to the next, and **terminates**, deciding when to stop or hand back to a person.

Never does: send the request (model interface), gather (input), present (context), or handle the response (output).

Control flow is the one primitive that wraps the others. How it arranges them decides what you've built:

Shape	Arrangement	Who decides what happens next
Single call	input → call → output, stop	nobody; it's fixed
Chat loop	call, show, hand back to the person, repeat	the person
Workflow	your code lays out the calls: draft, judge, rewrite	your code
Agent loop	call; while the model asks for tools, run them, send the results back, call again	the model

So the line between a *workflow* and an *agent* [4] is a fact about control flow: who decides the next step. Termination is half the definition, not an afterthought. A loop with no clear way to end is a bill with no clear way to end. The smallest agent loop ends when the model stops asking for tools. Bigger ones also stop on a step limit, a spending limit, a refusal, or a person saying stop.

3.5 Context

Does: before every call, **assembles** what the request holds and **fits** it into the space the model can read.

Never does: gather inputs (input), send the request (model interface), or decide when to call (control flow).

Every call starts from nothing. The model doesn't know where it's running, what day it is, who it is, what it did a minute ago, or what you told it last week, unless the harness puts it in the request. That's why so much of what people build into harnesses turns out to be context. These components are all ways of deciding what the model sees:

- **instructions:** the system prompt;
- **working memory:** this session;
- **episodic memory:** past sessions;
- **semantic memory:** facts that last;
- **procedural memory:** skills, recipes, playbooks;
- **retrieval:** search results put in the request;
- **compaction:** a summary in place of history that no longer fits;
- **self-knowledge:** the agent's own description, or its own source.

Input and context aren't the same thing. Input brings something to the edge of the request. Context decides whether it goes in, where, in what form, and what gets dropped to make room.

3.6 Summary

Primitive	Does	Never does
Model interface	request out, response back	decide when, gather, present, act
Input	gather from a person or the world	present, send, decide to go again
Output	show the response, or run a tool	decide whether results return, shape the next request
Control flow	sequence and terminate	send, gather, present, handle
Context	assemble and fit the request	gather, send, decide when

4 Rules for the hard cases

A taxonomy is only as good as its hard cases. The rules below settle the ones that come up. Each follows from the negative definitions in §3. Several were sharpened by the decomposition study in §7, and I mark those.

Place by what it does, not where it lives. Code that decides what the model sees is context, even inside a tool. Examples:

- opencode attaches nested AGENTS.md files to the Read tool’s result;
- Claude Code’s WebFetch runs a separate model call to shrink a page before the model sees it.

A guardrail decision is control flow even when it runs inside a tool’s execution, as Codex’s approval routine does. (*Sharpened by §7.*)

“Is a tool” doesn’t mean “is output.” In all three production harnesses, the model drives other primitives through tool calls: spawning a subagent, entering plan mode, writing a task list, searching for tools, scheduling a wake-up, asking for a new context window. The tool call is output. Its effect lands on the primitive it changes: control flow, context or input. (*From §7.*)

Backup or routing: ask what triggered the switch.

- **The request couldn’t be served** by the first model, because it was down, overloaded, or refused the request: switching to another model is the **model interface**. It only changes how one request gets a response.
- **The switch is keyed to the work:** this task is simple, this phase is planning. That’s **routing**, which is **control flow**, because it’s a decision about the work.

Some cases to check the rule against:

- *A fixed model for one kind of request*, such as compaction summaries, is a request setting. That’s the model interface.
- *Claude Code’s opusplan* uses one model to plan and another to execute. It’s keyed to the phase, so it’s control flow.
- *A content-classifier fallback* that retries a flagged request on another model is the model interface. Keeping the second model for the rest of the session is a separate, control-flow decision.

(*Sharpened by §7.*)

Streaming. *Receiving* a response as a stream is the model interface. *Printing* each piece as it arrives is output.

Parallelism. Running several *tool requests* from one response at once is output: it's how those requests get executed. Making several *model calls* at once, to fan out subtasks or to vote, is a way of arranging the work, so it's control flow.

Who reads it.

- Something kept **for the model**, such as an episode log the agent can search later, is **context**.
- Something kept **for the operator**, such as a trace of every call, is **observability** (§6).
- Something made **for the user**, such as a per-turn diff of changed files, is **output**: showing to a person.

The same file can serve two readers. Claude Code's session transcript is read by the operator and replayed by the harness on resume. In that case each use is placed separately. (*Sharpened by §7.*)

Who uses a person's answer.

- When the **harness** consumes the answer, as with “allow this command? [y/N]”, it's a **control-flow** decision, not new input.
- When the **model** consumes it, as when the model asks the person a question and gets the answer back as a tool result, it's **input** from a person.

(*From §7: Codex's `request_user_input`, opencode's `question tool`, Claude Code's `AskUserQuestion`..*)

A person acting directly. A shell command the *person* runs, such as `!cmd` in Codex, Claude Code and opencode, brings world state into the conversation. That's input, even though it reuses output's executor. (*From §7.*)

Programs the model writes. If the program sequences *model calls*, it's control flow, written at run time. Claude Code's workflow scripts are the example. If it only sequences *tools*, it's output: a script, however clever. The code modes in Codex and opencode run model-written JavaScript that calls tools without calling the model, so today they're output. (*From §7.*)

Bundles aren't parts. A *mode*, a *skill*, a *hook system* or an *agent profile* is packaging. Each bundles settings across several primitives. Split it and every piece lands in one:

- Plan mode is a context template, plus tools removed, plus an approval gate.
- A skill is procedural memory, plus scripts the model can run, plus a tool allow-list.
- A hook system is a dispatcher (control flow) whose events each touch one primitive.

(*From §7.*)

The “augmented LLM.” The building block of a model with retrieval, tools and memory [4] isn't a primitive. It splits into context (retrieval, memory) and output (tools) around a model interface.

Tool protocols. MCP [5] is mostly output you plug in: tools someone else built, behind one protocol. Its other features land elsewhere:

- resources and prompts decide what the model sees, so they're **context**;
- *sampling* borrows the client's **model interface**;
- *elicitation* asks the person, which is **input**.

(*Sharpened by §7.*)

Multiple agents. One agent handing a task to another is a shape of control flow: the outer harness sequencing an inner one. Delegation adds no new primitive. It nests harnesses. The same rule covers model-backed parts *inside* a harness: an approval reviewer, a stop judge, a memory extractor. Each is control flow plus a model-interface call.

5 Building it back up: the five are enough

A taxonomy can be complete on paper and still miss something in practice. The test is to build an agent from nothing but the five and see whether anything else is needed. That’s the core of this paper, and the first four lessons of the artifact.

The build goes outward from the model, in a different order from §3:

- the model interface comes first, because nothing reaches the model without it;
- each later step adds the primitive the last one was missing.

The example is quark, my own agent. Each step’s `quark.py` is the last step’s file plus that primitive and nothing else, and all four files are in Appendix A; the runs quoted below are in Appendix B. It’s one way to build each primitive, not the only way.

5.1 The model interface alone (9 lines)

The first harness is one call. It sends a hard-coded question, “*What’s in this directory?*”, and prints the whole response. The response has three fields that matter:

- `content`: the tokens out, as blocks;
- `stop_reason`: why it stopped;
- `usage`: tokens in and out.

The reply is the lesson. The model says it can’t see the directory and suggests running `ls`. **It knows the right next step and can’t take it.** Both ends are stubs: the input is a string no one typed, and the output is a dump that handles nothing.

5.2 Input and output (21 lines)

Input and output are built separately, then put together.

- **Input** takes a task from the command line and describes one tool, `bash`, in the request. Given “*how many lines are in README.md?*”, the model answers the only way it can: a `tool_use` block asking for `wc -l README.md`, with `stop_reason: tool_use`. Output is still a stub, so nothing runs it. The model asked; nothing acted.
- **Output** handles each block of the response. It prints text, and it runs a tool request with `subprocess`.
- **Together** they need one more piece neither had alone: the command’s result, wrapped as a `tool_result`. That’s input from the world.

The joined harness runs `wc -l` and prints `71 README.md`. **The model never sees the 71.** It sits in a list, and nothing sends it. The path ends at output.

5.3 Control flow (30 lines)

Step 2 goes inside `while True`. The response and the results are appended to `messages`. If the model asked for anything, the loop calls again. When it stops asking, the run ends, or, in a chat, it hands back to you.

Asked which lesson’s `quark.py` is longest, the agent ran one `find ... | wc -l` and answered from what it found. That’s two calls, and the second only happened because the first one’s result went back.

That’s an agent now: the model decides what happens next. The same four pieces, arranged differently, make a single call, a chatbot or a workflow (§3.4). The artifact’s second example for this step is a workflow, an evaluator–optimizer, where your code decides the order of the calls.

But it knows nothing: not where it is, not what day it is, not who it is, not what you told it yesterday. And `messages` grows every pass, so a long session will outgrow what the model can read. Something has to decide what the model sees.

5.4 Context (48 lines)

The last primitive adds five context components:

- **Working memory.** Step 3’s `messages`, renamed for what it is.
- **Instructions.** A system prompt written as models of self, world, other selves and body. It’s rebuilt every call, so the directory and the date are current.
- **Self-knowledge.** quark’s own source file, put in that prompt so the model can see the harness it runs in.
- **Semantic memory.** A file the agent reads and writes with the tool it already has. There’s no memory code at all.
- **Reactive compaction.** When the API refuses a request as too long, the oldest turns are dropped and the rest summarized.

The runs show it working:

1. **Asked what it is**, quark describes its name, its body, its loop and its memory file, because they’re in its context.
2. **Told “*remember that I prefer short answers*”**, it writes that to its memory file.
3. **In a new session**, with empty working memory, asked what it knows about me, it reads the file and answers.

Working memory carried nothing between the runs. Semantic memory did.

5.5 What the build shows

Step	Adds	quark.py	What’s still missing
1	Model interface	9 lines	Both ends are stubs. The model says the next step is <code>1s</code> , and nothing can run it.
2	Input and output	21 lines	A tool runs, but its result never reaches the model.
3	Control flow	30 lines	The result goes back, but the agent knows nothing about itself, its world or its past, and its history grows without limit.
4	Context	48 lines	Nothing. It works, remembers, knows what it is, and compacts when it’s full.

Each step’s missing piece is the next primitive, and no step needs anything outside the five. The finished harness is in Appendix A.

Each primitive also gets a fuller second example, to show that a primitive is a space of choices, not one design:

- **Model interface:** timeouts, retries, a backup model, streaming and adjustable thinking.
- **Input and output:** input and output through a Telegram chat, with two tools run at the same time, timeouts, and an incomplete request that isn't run.
- **Control flow:** a step limit and refusal handling, plus the evaluator-optimizer workflow.
- **Context:** an episode log, skill files, token counting before each call, and trimming of long tool results.

None of them needed a sixth primitive.

6 Production hardening folds back into the primitives

A harness that works isn't one you'd run unattended. Production agents add tracing, approvals, sandboxes, retries, caching and evaluation. Each has its own vendors and its own literature, so it's natural to treat each one as a new building block. The primitives say otherwise:

Corollary. Production concerns add *hardening*, not new primitives. Each piece of a production layer lands inside a primitive, where that primitive already acts.

The artifact's second part tests this one layer at a time. The code each layer adds is in Appendix C:

- Each layer's `quark.py` is the previous one plus that layer.
- Each names the primitives it's built on.
- Each ends with a “*Notice what [layer] never does*” paragraph naming the primitives it leaves alone.

Layer	Folds into	Why there	Mechanism	Lines added (<code>quark.py</code>)
Observability	control flow	the only primitive that sees the whole sequence	read the clock, <code>usage</code> and exit codes that are already there; append one JSON line per event	+12 (59)
Guardrails	control flow (mostly)	control flow already decides whether a request becomes a command, and whether to go again	allow, ask or deny before each tool; a refusal returned as a readable tool result; step, token, cost and repeat limits; an interrupt	+26 (83)
Sandboxing	output	<i>where</i> a tool runs was always output's choice	commands run by <code>docker exec</code> in a container with no network, capped resources, a read-only root and only the working folder mounted, so the kernel enforces the limits, not a check on the text	+13 (94)
Resilience	model interface, output, context	where the world can say no, and what must be replayed	retry transient failures with backoff, fall back to a backup model, stop cleanly; answer every tool request even when it's broken; save each step and resume, marking an interrupted command as <i>may</i> or <i>may not have run</i>	+53 (135)

Layer	Folds into	Why there	Mechanism	Lines added (quark.py)
Performance	context, model interface, output	where the tokens and the waiting are	cache marks and trimming (context); streaming and a fixed small model for summaries (model interface); tools run at the same time (output)	+47 (159)
Evaluation	outside the harness	it treats the agent as a box	an outer loop that sets up a case, runs the real agent, grades what it left behind, and compares with the last run	+38 (196)

A few rows need more than a table cell.

Some layers span several primitives without adding one.

- **Resilience.** A retry happens inside one call, so it’s the model interface. Answering a broken tool request is output. Resuming a run replays saved working memory, which is context. Codex does the same at request-assembly time: it fills in “aborted” results for tool calls that never finished.
- **Performance.** Running the model’s tool requests together is output. A cache mark is a choice about how the request is laid out, so it’s context. Performance’s fuller example also routes each task to a model tier, and that part is control flow, by the routing rule in §4.

Guardrails mostly sit in control flow, but not entirely. In opencode and Claude Code, denied tools are also removed from the request, and that slice is context. The decision still has one home; the enforcement can reach into another primitive.

Layers aren’t independent of each other. I first wrote that each layer “leaves the other primitives unchanged.” The production harnesses show that’s too strong. In Codex, the approval policy and the sandbox policy feed each other:

1. The approval requirement is computed from the sandbox setting.
2. An approval changes how the command runs.
3. A sandbox denial triggers a new approval.

Claude Code is the same: its sandbox can stand in for a permission prompt, and the model can ask to leave the sandbox, which sends that request back through the guardrails. Each piece still lands in one primitive. What the study shows is that layers can read each other’s state. The primitives stay disjoint; the layers don’t stay separate.

Evaluation is the exception that proves the rule. It sits in none of the five, because it stands outside the agent. But it’s built *from* them:

- an outer **control-flow** loop;
- the task handed in as **input**;
- the check run as **output**;
- a model grader reached through the **model interface**.

It’s a second harness wrapped around the first. In the artifact it caught a change that looked harmless (Appendix C.6). Lowering the step limit from 20 to 1 kept three of four cases passing and flagged the fourth as `REGRESSED`. In the production harnesses, evaluation is just as clearly outside:

- **Codex and opencode** ship only engineering tests, against a mocked or recorded model.
- **Claude Code’s** `claude plugin eval` grades plugins and skills by running the real agent.

So the production harness of 196 lines reads under the same five headings as the 48-line one. The layers make the primitives sturdier; they don't add to them. I show the corollary by cases, not by proof, and §9 says what would refute it.

7 Taking apart production harnesses

If the primitives are right, real harnesses should sort under them, including big ones that weren't written to teach anything. I took apart three production coding agents, part by part, with the checklist the artifact ends on:

```
control flow      what kind of loop? who decides when to stop?
|-- input        where do inputs come from: people, the world, both?
|-- context      what does the request hold? which memories? how does it fit?
|-- model interface where does the request go, and how does the response come back?
'-- output       where do outputs go? which tools, and how are they run?
```

7.1 Method

For each harness:

1. I mapped the modules on the request path.
2. I assigned every meaningful part to exactly one primitive, or to a production layer with the primitive it folds into, with evidence and a reason from the definitions in §3.
3. I listed every part that was hard to place, and asked of each: does it fit none of the five?

Harness	Source	Version
OpenAI Codex CLI [6]	source code, github.com/openai/codex (Rust, codex-rs/)	commit 7ac954e, 2026-10-06
opencode [7]	source code, github.com/anomalyco/opencode (TypeScript; formerly sst/opencode)	commit 652c090, v1.18.34, 2026-10-06
Claude Code [8]	public documentation only (code.claude.com/docs and Anthropic engineering posts); its source isn't published	docs as of 2026-10-06 (CLI v2.1.288)

The work was done with AI assistance, in my own way of working: an agent read the code or docs with the paper's definitions in hand and produced the assignments with file-and-line or URL evidence, so every assignment can be checked against its source. The full tables, with the evidence for every assignment, are in the supplement.

7.2 What each harness chose

	Claude Code	Codex	opencode
Control flow	Agent loop that’s also a chat loop: you can interrupt, and messages typed mid-turn are queued into the same turn. Stops: no tool calls, <code>maxTurns</code> , <code>maxBudgetUsd</code> , a refusal, a Stop-hook veto. Nested: subagents (3 deep, 20 at once), agent teams, model-written workflow scripts, a model-judged <code>/goal</code> , schedules.	Agent loop inside a chat loop. Stops: no tool call, a Stop hook, an error, an interrupt, a session token budget. No step limit. Nested: review mode, sub-agents, an LLM approval reviewer (“guardian”).	Agent loop with your turns between runs. Stops: natural finish, structured output, content filter, a denied permission, cancel. The step limit is soft : it tells the model to stop, but still sends the tools. Nested: subagents via a task tool.
Input	Terminal, IDE, web, Slack, <code>-p</code> , stdin, the SDK stream; tool results; background-task and channel events; schedules; questions the model asks you.	TUI, headless <code>exec</code> , a JSON-RPC app server, voice; text, images, audio, mentions; mid-turn steering and an inter-agent mailbox; tool results; <code>!cmd</code> .	One HTTP server fed by the TUI, the CLI, editors (ACP), web and desktop apps, a GitHub Action, Slack; <code>@files</code> , images, MCP resources, slash commands, <code>!cmd</code> ; tool results; language-server diagnostics after edits.
Context	System prompt and output style; a CLAUDE.md hierarchy (managed, user, project, local); a model-written memory file; skills loaded on demand; search on demand instead of an index; reminders; clear-then-summarize compaction; tool search; truncation inside tools; a cache-ordered layout.	Model-specific base instructions; AGENTS.md from root to cwd; an environment block re-sent only when it changes; working memory with truncated tool outputs; local or server-side compaction before, during and after a turn; cross-session memories; skills; deferred tool loading.	A system prompt per model family; an environment block; AGENTS.md and CLAUDE.md; skills; mode reminders; proactive and reactive compaction; pruning old tool outputs; truncation that spills to a file; cache marks.
Model interface	Anthropic SDK over four providers or a gateway; streaming; retries with a timeout; a fallback chain for outages; a small model for background calls; effort, thinking and cache settings.	Responses API only; WebSocket with an HTTP fallback; always streamed; retries with backoff and jitter; OpenAI, Bedrock, Ollama and LM Studio; effort and service tier. No backup model for turns.	Vercel AI SDK over about 20 bundled providers; always streamed; retries with backoff; a small model for titles. No backup model.
Output	Many specific tools, with Bash as the general fallback; read-only tools in parallel and writes in sequence; background commands; MCP; server tools (web search); terminal, JSON or stream-JSON.	Unified PTY exec, <code>apply_patch</code> , plan, image and MCP tools, hosted web search, a JavaScript code mode; tools start as soon as each request finishes streaming; a lock lets safe tools run in parallel.	bash, read, edit, write, patch, grep, glob, task, web, todo, skill, question tools, plus MCP and plugins; run concurrently by the SDK; a code mode; shown through the server’s event stream.

The three make very different choices inside each primitive: how they stop, where input comes from, how context fits, how tools run. Every one of those choices is a choice *inside* a primitive.

7.3 Production layers

Layer	Claude Code	Codex	opencode
Observability	✓ OpenTelemetry metrics, events and traces; transcripts; cost	✓ OpenTelemetry, tracing spans, a trace bundle	✓ structured logs, OpenTelemetry spans, per-step tokens and cost
Guardrails	✓ six permission modes, allow/ask/deny rules, hooks, a learned auto-approval classifier, turn and budget caps	✓ approval policies, a rule language for commands, an LLM reviewer, hooks, a token budget	✓ allow/ask/deny rules per agent, a doom-loop check (the same call three times), a subagent depth limit
Sandboxing	✓ Seatbelt or bubblewrap with a network proxy (off by default); worktrees; cloud VMs	✓ Seatbelt; bubblewrap, seccomp and Landlock; Windows restricted tokens; a network proxy	✗ no kernel limits: permission prompts, worktrees, and a confined interpreter for code mode only
Resilience	✓ retries, a fallback chain, snapshots before edits, resume and fork	✓ retries, a transport fallback, aborted-tool answers, resume and fork from a saved log	✓ retries, interrupted tools still answered, every part saved as it streams, snapshots and revert
Performance	✓ prompt caching, tool search, streaming, parallel reads	✓ streaming, incremental WebSocket requests, a cache key, environment diffs, parallel tools	✓ cache marks, pruning and truncation, concurrent tools
Evaluation	partial: <code>claude plugin eval</code> , outside the agent, for plugins and skills	✗ only engineering tests against a mocked model	✗ only engineering tests with recorded model responses

Every layer in §6 shows up in at least two of the three. Each one lands where the corollary says it should. The one layer missing from all three cores, evaluation, is the one the paper puts outside the harness.

7.4 What was hard to place

The study’s job was to find something that doesn’t fit, so these are the parts that pushed hardest, and how they settled:

1. **Hooks** (Claude Code, Codex, opencode plugins). A hook system can block a tool, add context, rewrite a tool’s arguments, veto a stop, or log. It’s a bundle, not a part. The dispatcher is control flow, and each event’s effect lands on one primitive.
 - One effect stayed a judgment call: Claude Code’s `updatedInput`, which rewrites the model’s tool arguments before they run. It handles the response, so I place it in output. But it’s used as a guardrail.
2. **The guardrail–sandbox coupling** (Codex, Claude Code). These are the strongest pressure on §6, and they’re why I rewrote its claim. They don’t add a primitive.
3. **Model-backed parts inside the harness:**
 - Claude Code’s auto-approval classifier and its `/goal` judge;
 - Codex’s guardian reviewer;
 - Codex’s memory extraction.

§9.1 predicted that learned components would test the boundary. All four settled as the paper reads them: control flow plus a model-interface call, which is a nested harness.

4. Programs the model writes:

- Claude Code’s workflow scripts sequence agents, so they’re control flow written at run time.
- The code modes in Codex and opencode run model-written JavaScript that calls tools with no model call in between, so they’re output.

This was the closest case to a break. It settled once the rule in §4 was stated: a program is control flow when it sequences model calls.

5. **Undo.** Claude Code’s “restore code” and opencode’s revert roll back files at the person’s request, not the model’s. Snapshotting before each edit is resilience in output, and deleting messages is context. But the file restore itself isn’t *handling the model’s response*. I place it as resilience over output’s past actions. It’s the weakest fit in the study, and §9.2 discusses it.
6. **Server-side parts of the harness.** These run on the provider’s side of the boundary:

- Claude Code’s server-side classifier review;
- Codex’s server-side compaction;
- the provider rerouting a request to a different model.

Each still places: a guardrail, context, and a model-interface event shown to the person. But the parentheses in *Agent = Harness(Model)* aren’t a clean client/server line.

7. **A remote agent behind the model interface.** opencode can treat GitLab’s Duo Workflow service as a “language model”. That service runs its own agent loop and calls opencode’s tools. It settles only as a nested harness whose interface is a remote agent, not a model.
8. **Persistence that’s also a trace.** Each harness’s saved transcript is read by the operator (observability) and replayed on resume (context). It’s one artifact with two readers, and each use is placed separately.

Verdict. Every runtime part of all three harnesses does one of three things:

- lands in one primitive;
- splits into pieces that each land in one; or
- nests a harness.

I found nothing that needs a sixth primitive. The study did change the paper. It made me state five rules I had been using without saying so: place by function, the tool channel, who reads, who uses an answer, and model-written programs. It also made me weaken one claim, that layers leave each other alone.

7.5 Products

The same holds for products. Each one *centres on* a primitive or a layer, and most reach into others:

Product	Centres on	Also touches
LiteLLM, OpenRouter (gateways) [9, 10]	the model interface: one API over many providers, load balancing, failover to backup models, retries, rate-limit handling	spend limits (guardrails), logging (observability), response caching (performance); OpenRouter’s Auto Router picks a model per prompt, which is routing, so control flow

Product	Centres on	Also touches
MCP [5]	output you plug in: tools behind one protocol	resources and prompts (context), sampling (model interface), elicitation (input)
Mem0 [11]	context: store what to remember, return what’s relevant per request	its extraction is its own model call
LangGraph [12]	control flow: steps and edges laid out as a graph and run	its checkpointer serves resilience, working memory (context) and approval pauses (guardrails)
Langfuse, LangSmith [13, 14]	observability, so built on control flow	evaluation; versioned prompts served at run time (context)
Open Policy Agent [15]	the decision part of guardrails; the harness still enforces it	none
NeMo Guardrails [16]	guardrails in control flow: allow or block around each call	input and retrieval rails that edit what the request holds (context)
E2B, Daytona, Modal Sandboxes [17–19]	sandboxing, so built on output	snapshot and resume (resilience)
Temporal [20]	resilience: record each step, retry it, replay to resume	it runs your workflow code, so it also hosts control flow
Braintrust, promptfoo, Inspect [21–23]	evaluation: run cases, grade them, keep the history or logs	production tracing (Braintrust); Inspect also supplies the agent under test

So products are made of primitives too. They’re sold as one thing and built from several.

8 Case study: the harness that wrote its own lessons

I used the finished 48-line harness for real work on the companion artifact, the repository that teaches it. The evidence quoted below is in Appendix D. Claude Code operated it from a terminal, the way a person would, and quark never saw Claude Code. Over four runs, quark:

1. read the whole repository and wrote what it learned to its memory file, about 25 KB of notes, one entry per lesson;
2. in a new session with empty working memory, wrote slide decks for the four primitive lessons *from memory alone*, without opening a lesson file;
3. wrote the six production lessons, one session per lesson. Each built its `quark.py` from the one before, ran its code for real, and pasted in the real output;
4. in another new session, wrote slide decks for the six production lessons, again from memory.

The setup is itself an example of the claim. The script that ran one quark session per lesson, stopping if a lesson produced no `quark.py`, is control flow one level up: a workflow around an agent.

The failures are the interesting part, and each lands on a primitive.

A silent stop, then a crash: model interface and output. The session writing the observability lesson stopped twice without writing anything. A trace of each response’s `stop_reason` showed why: with thinking on, a 4,096-token cap wasn’t enough to think and then write a file. (Adding that trace was observability, put in for diagnosis.) On the third try, a response hit `max_tokens` halfway through a tool request, and the harness crashed with `KeyError: 'cmd'`. That’s the incomplete request §3.3 lists under “when not to run.”

- **The fix** was a request setting: `max_tokens` raised to 16,384 across the artifact.
- **The guard** against the same crash, refusing to run an incomplete request, is part of what the resilience layer adds to output.

The agent killed itself: output, guardrails and sandboxing. To test crash recovery, quark killed its test agent with `pkill -9 -f 08-resilience/quark.py`. Its own command line contained that path, so it killed itself too. Its Docker container outlived it: the leftover the sandboxing lesson had warned about for a harness killed hard.

- **The fix** was a rule in the prompt: stop processes by PID, never by name. A guardrail could enforce that rule as policy.
- **Claude Code**, operating quark, then made the same mistake with its own shell command. The hazard is in how the action is written, not in one agent.

Confident fabrication from memory: context, caught by evaluation. The slides were written from memory, and memory is a summary.

- **The primitive decks** needed 22 slides tightened but had nothing invented.
- **The production decks** needed 47 slides corrected and 5 removed. Writing from memory, quark had filled gaps with runs that never happened, cache numbers no run produced, and a reversed account of which model was benched. Other slides simplified the code until they misstated it, or put a mechanism under the wrong primitive.

That’s a context failure. What the request held was a lossy summary, and the model couldn’t tell it from fact. Only review caught it, checking each slide against its source, which is evaluation done by hand.

What it shows. The 48-line harness did real work over many sessions: it read, wrote, ran and debugged code and documents. Every failure could be placed under one primitive or one layer, and they were the failures the artifact had already described in §3 and §6. So the vocabulary worked for diagnosis, not just for description. This is one case, and the agent and I are both inside it. It shows the framework in use; it isn’t independent evidence.

9 Discussion

9.1 What would refute this

I wrote the claim so it can be shown wrong. It fails if someone finds a part of a working harness that meets all three of these conditions:

1. it fits none of the five definitions;
2. it can’t be read as hardening inside one or more of them;
3. it isn’t a harness nested inside another (§4).

The artifact ends with that invitation: if you find something that genuinely fits none of the five, I’d like to hear about it.

Before the study, I named three places the boundary was most likely to break:

- **learned components inside the harness;**
- **search or decoding loops;**
- **harnesses whose control flow is written by a model at run time.**

The production harnesses supplied live cases of the first and third, and all of them held:

- Claude Code’s approval classifier and `/goal` judge;

- Codex’s guardian reviewer;
- Claude Code’s workflow scripts;
- the code modes in Codex and opencode.

Search or decoding loops didn’t come up and remain untested.

9.2 Where the definitions strain

Nothing in the study needed a sixth primitive. Two definitions do strain.

- **Output and actions a person starts.** Output is defined around *the model’s* response. A person who undoes the agent’s file changes (Claude Code’s restore, opencode’s revert) is having the harness act on the world without a model response. I place it as resilience over output’s past actions. A cleaner fix might widen output to *the harness acting on a person or the world, usually at the model’s request*. I’ve left the narrower definition because it’s the one the artifact teaches. This is the first place to look for a refinement.
- **Model switches with mixed triggers.** The backup-or-routing rule settles a switch by what triggered it. A switch that starts as a failure and then persists, like Claude Code’s content-classifier fallback, is two decisions: model interface, then control flow. That’s consistent, but it shows that one feature can sit across a boundary.

9.3 Limitations

- **The core evidence is mine.** The build, the production layers and the case study come from my own artifact and my own agent. §7 adds three outside systems, but they’re three, all coding agents. Harnesses for other jobs, such as voice, browsing or robotics, haven’t been tested.
- **Claude Code was read from its docs.** I could only check what Anthropic says it does. That’s weaker than reading the code, especially for the “never does” half of each definition.
- **The decompositions were AI-assisted.** Each assignment cites its evidence so anyone can check it, but a second, independent decomposer would make the result stronger.
- **Some boundaries are rules I chose.** Backup or routing, receiving or printing, who reads, who uses an answer: these follow from the definitions, but each had to be stated. Someone with different definitions could draw the line elsewhere. The value is that every choice is written down.
- **The model is held fixed.** I set model development aside. Fine-tuning a model on a harness’s traces moves work from one part to the other, and I don’t analyze that trade.

9.4 Implications

- **A shared vocabulary.**
 - “Our agent uses memory” becomes a claim about context: which components, assembled how, and fitted how.
 - “Our agent is reliable” becomes claims about resilience in the model interface and output, and about evaluation outside them.
- **Something to disclose.** Calls to disclose the harness when comparing agents [2] need a schema for what to disclose. The five primitives and the layers are one. §7.2 is what such a disclosure looks like for three agents.

- **Buying vs. building.** Every product in §7.5 replaces part of a primitive or a layer. Choosing one is choosing which part of your harness you can no longer see.

10 Related work

Cognitive architectures. CoALA [24] organizes language agents into memory modules, an action space (internal and external), and a decision-making cycle. It draws on cognitive science and symbolic AI, and it’s the closest earlier attempt to say what agents are made of. Its categories come from cognition; mine come from what harness code does to a request and a response.

They line up like this:

- CoALA’s **memory modules**, and the **retrieval and learning actions** that read and write them, are context here.
- Its **reasoning actions** are calls through the model interface.
- Its **external actions** are output, with input bringing back their results.
- Its **decision cycle** is control flow.

The model interface as a primitive, and the production layers as a category, have no direct counterpart in CoALA.

Agent primitives for multi-agent systems. “Primitives” means something different there, so the distinction needs drawing. Jin et al. [25] break multi-agent systems into recurring latent patterns: review, voting and selection, planning and execution. The patterns pass information through the KV cache instead of text, and an organizer composes them for each query. Their primitives are patterns *of* model calls. In my terms they’re shapes of control flow, the organizer included. Passing state as KV cache is a choice that sits at the boundary of context and the model interface. The two views are complementary.

Workflows and agents. Schluntz and Zhang [4] distinguish workflows from agents by who decides the path. They name five workflow patterns built on an “augmented LLM”: prompt chaining, routing, parallelization, orchestrator–workers and evaluator–optimizer. I adopt their workflow/agent line as a property of control flow, and I split the augmented LLM into context, output and the model interface (§4).

Harness engineering. Two papers supply this one’s premise, and one of them also gives it an outside check.

- **Zhang et al.** [2] argue that harness configuration can drive more performance variance than model choice. They propose a standard for disclosing the harness. Their argument is the premise of this paper: if the harness matters that much, it needs a vocabulary. §9.4 offers the five primitives as the schema their disclosure needs.
- **Fan et al.** [3] hold a coding harness’s loop fixed and vary planning, the action space and context management across four models. Each choice changes the agent’s trajectories in a different way. They chose those three parts on their own terms, and each lands inside one of the five here:
 - planning is control flow;
 - the action space is output;
 - context management is context.

That’s an independent decomposition falling inside this one. Their result also supports the claim in §7.2 that harnesses differ by the choices made *inside* each primitive.

11 Conclusion

An agent is a model wrapped in a harness. A harness is five things: input, context, the model interface and output, with control flow sequencing them and deciding when to stop.

The build. One at a time, outward from a single API call, the five make a working agent in 48 lines, and nothing else is needed. That’s the claim.

Production hardening. Everything a production harness adds after that, from observability to evaluation, folds back into the same five.

Real systems. Three production coding agents, built by three different teams, sort under the five too. The study made the rules sharper, and it found nothing that needs a sixth.

The point isn’t that every harness should look like quark. It’s that every harness, however big, can be read with five questions, and every design choice in it belongs to one of them. Five primitives are all you need to build an agent harness, and all you need to take one apart.

Acknowledgments and disclosure of AI use

The companion artifact was built with two AI agents, and the paper says so because the case study depends on it.

- **Claude Code** (Anthropic) worked with me on the artifact’s README and primitive lessons. It operated quark and reviewed quark’s output. It ran the decomposition study in §7 under my direction, and it helped draft this paper from the artifact’s content.
- **quark**, the agent built in §5, running on Anthropic’s Claude models, wrote the six production lessons and the slide decks. Those were then reviewed and corrected as described in §8.

The thesis, the five primitives, their definitions, and the method are mine, and I’m responsible for every claim in this paper. Every run quoted here is excerpted in the appendices and reproduced in full, with transcripts, in the artifact.

References

- [1] Chase Dovey. harness-engineering: An open course with runnable code (companion artifact). <https://github.com/averagejoeslab/building-agents>, 2026. Average Joes Lab.
- [2] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026. Position paper; listed at ICML 2026 (venue and track to confirm).
- [3] Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. *arXiv preprint arXiv:2609.20804*, 2026.
- [4] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering Blog, December 19, 2024, December 2024. URL <https://www.anthropic.com/engineering/building-effective-agents>.
- [5] Model Context Protocol. Model Context Protocol: Architecture overview. <https://modelcontextprotocol.io/docs/learn/architecture>, 2026. Accessed 2026-10-06.
- [6] OpenAI. Codex CLI. <https://github.com/openai/codex>, 2026. Commit 7ac954e, 2026-10-06.

- [7] opencode contributors. opencode. <https://github.com/anomalyco/opencode>, 2026. Commit 652c090, v1.18.34, 2026-10-06.
- [8] Anthropic. Claude Code documentation. <https://code.claude.com/docs>, 2026. Accessed 2026-10-06.
- [9] BerriAI. LiteLLM router: Load balancing and fallbacks. <https://docs.litellm.ai/docs/routing>, 2026. Accessed 2026-10-06.
- [10] OpenRouter. OpenRouter: Model fallbacks and auto router. <https://openrouter.ai/docs/guides/routing/model-fallbacks>, 2026. Accessed 2026-10-06.
- [11] Mem0. Mem0: Memory operations. <https://docs.mem0.ai/core-concepts/memory-operations/add>, 2026. Accessed 2026-10-06.
- [12] LangChain. LangGraph: Persistence. <https://docs.langchain.com/oss/python/langgraph/persistence>, 2026. Accessed 2026-10-06.
- [13] Langfuse. Langfuse. <https://github.com/langfuse/langfuse>, 2026. Accessed 2026-10-06.
- [14] LangChain. LangSmith: Observability. <https://docs.langchain.com/langsmith/observability>, 2026. Accessed 2026-10-06.
- [15] Open Policy Agent. Open Policy Agent documentation. <https://www.openpolicyagent.org/docs>, 2026. Accessed 2026-10-06.
- [16] NVIDIA. NeMo Guardrails: Guardrail types. <https://docs.nvidia.com/nemo/guardrails/latest/about/rail-types.html>, 2026. Accessed 2026-10-06.
- [17] E2B. E2B documentation. <https://e2b.dev/docs>, 2026. Accessed 2026-10-06.
- [18] Daytona. Daytona: Sandboxes. <https://www.daytona.io/docs/en/sandboxes/>, 2026. Accessed 2026-10-06.
- [19] Modal. Modal: Sandboxes. <https://modal.com/docs/guide/sandboxes>, 2026. Accessed 2026-10-06.
- [20] Temporal. Temporal: Event history. <https://docs.temporal.io/encyclopedia/event-history>, 2026. Accessed 2026-10-06.
- [21] Braintrust. Braintrust: Evaluate. <https://www.braintrust.dev/docs/evaluate>, 2026. Accessed 2026-10-06.
- [22] promptfoo. promptfoo: Introduction. <https://www.promptfoo.dev/docs/intro/>, 2026. Accessed 2026-10-06.
- [23] UK AI Security Institute. Inspect. <https://inspect.aisi.org.uk/>, 2026. Accessed 2026-10-06.
- [24] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=1i6ZCvflQJ>. arXiv:2309.02427.
- [25] Haibo Jin, Peng Kuang, Ye Yu, Xiaopeng Yuan, and Haohan Wang. Agent primitives: Reusable latent building blocks for multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03695.

A The build, file by file

These are the four files of §5, in full. Each is the one before plus one primitive.

A.1 The model interface alone (9 lines)

```
from anthropic import Anthropic

client = Anthropic()
reply = client.messages.create(
    model="claude-sonnet-5-5",
    max_tokens=16384,
    messages=[{"role": "user", "content": "What's in this directory?"}],
)
print(reply.model_dump_json(indent=2))
```

A.2 Plus input and output (21 lines)

```
import subprocess, sys
from anthropic import Anthropic

client = Anthropic()
tools = [{"name": "bash", "description": "Run shell command -- the whole system is in reach", "input_schema":
    {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}
task = " ".join(sys.argv[1:]) or input("> ")
reply = client.messages.create(
    model="claude-sonnet-5-5",
    max_tokens=16384,
    tools=tools,
    messages=[{"role": "user", "content": task}],
)
results = []
for block in reply.content:
    if block.type == "text":
        print(block.text)
    if block.type == "tool_use":
        print(f"${block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.
            STDOUT, text=True)
        print(done.stdout)
        results.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {
            done.returncode)}")})
```

A.3 Plus control flow (30 lines)

```
import subprocess, sys
from anthropic import Anthropic

client = Anthropic()
tools = [{"name": "bash", "description": "Run shell command -- the whole system is in reach", "input_schema":
    {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}

task = " ".join(sys.argv[1:]) or input("> ")
chat = len(sys.argv) < 2
messages = [{"role": "user", "content": task}]

while True:
    reply = client.messages.create(model="claude-sonnet-5-5", max_tokens=16384, tools=tools, messages=
        messages)

    results = []
```

```

for block in reply.content:
    if block.type == "text":
        print(block.text)
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.
STDOUT, text=True)
        print(done.stdout)
        results.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit
{done.returncode})"})

messages.append({"role": "assistant", "content": reply.content})
if results:
    messages.append({"role": "user", "content": results})
    continue
if not chat or (task := input("\n> ")) == "/q":
    break
messages.append({"role": "user", "content": task})

```

A.4 Plus context: the finished harness (48 lines)

This is the full 48-line harness from §5, with the system prompt shortened to The full prompt is one long line in the artifact’s `lessons/04-context/quark.py`; it holds the self, world, other-selves and body models described in §5.4, and ends with the file’s own source. The comments mark which primitive each part belongs to; they aren’t in the original file.

```

import subprocess, sys, os, datetime
from anthropic import Anthropic, BadRequestError

client = Anthropic() # model interface
tools = [{"name": "bash", "description": "Run shell command -- the whole system is in reach", "input_schema":
{"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}] # output
def mechanics(): return "\n".join('def system(): return "<system prompt redacted so you can see your self
mechanics in harness>"' if l.startswith("def system():") else l for l in open(__file__).read().split("\n"
)) # context: self-knowledge
def system(): return [{"type": "text", "text": f"# Self Model\n\n**Identity:** You are quark ... **Where:** {
os.getcwd()}\n\n**When:** {datetime.date.today()} ... ``python\n{mechanics()}\n``", "cache_control": {"
type": "ephemeral"}}] # context: instructions

def compact(working_memory, drop): # context: compaction
    turns = [i for i, m in enumerate(working_memory) if m["role"] == "user" and isinstance(m["content"], str)
    ]
    if drop > len(turns): sys.exit("[working memory can't be summarized small enough]")
    keep = working_memory[turns[drop]:] if drop < len(turns) else [working_memory[turns[-1]]]
    summary = client.messages.create(model="claude-sonnet-5-5", max_tokens=2048, system=system(), messages=
keep + [{"role": "user", "content": "Your working memory is full. Summarize into a gist that preserves
what matters for continuing."}])
    gist = next((b.text for b in summary.content if b.type == "text"), "")
    return [{"role": "user", "content": f"[your prior working memory, summarized] {gist}"}]

task = " ".join(sys.argv[1:]) or input("> ") # input: from a person
chat = len(sys.argv) < 2
working_memory, drop = [{"role": "user", "content": task}], 0 # context: working memory

while True: # control flow
    try:
        if drop:
            working_memory, drop = compact(working_memory, drop), 0
            reply = client.messages.create(model="claude-sonnet-5-5", max_tokens=16384, system=system(), tools=
tools, messages=working_memory) # model interface
        except BadRequestError as e:
            if "prompt is too long" not in str(e): raise
            drop += 1
            continue

```

```

results = []
for block in reply.content:
    if block.type == "text":
        print(block.text)
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.
STDOUT, text=True)
        print(done.stdout)
        results.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit
{done.returncode})"}) # input: from the world

working_memory.append({"role": "assistant", "content": reply.content})
if results:
    working_memory.append({"role": "user", "content": results})
    continue
if not chat or (task := input("\n> ")) == "/q":
    break
working_memory.append({"role": "user", "content": task})

```

B The runs quoted in §5

Each excerpt is from one real run. The model’s wording varies from run to run.

B.1 The model interface alone. The response’s `content` holds an empty `thinking` block and a `text` block. The text begins:

```

I can't see your file system, so I can't tell what's in your directory. No files or tools have been shared in
this conversation. Here's how you can check yourself:

**macOS / Linux (Terminal)**
ls          # basic listing

```

`stop_reason` is `end_turn`, and `usage` shows 15 input tokens and 331 output tokens, 38 of them `thinking`.

B.2 Input, with output still a stub. Given “*how many lines are in README.md?*”, the whole `content` is one tool request:

```

{"id": "toolu_01UksmSeN8qGMiYrrgvUDZC", "input": {"cmd": "wc -l README.md"}, "name": "bash", "type": "
tool_use"}
"stop_reason": "tool_use"

```

B.3 Input and output together. The same task. The command runs, and its result never reaches the model:

```

$ wc -l README.md
71 README.md

```

B.4 Control flow. “*which quark.py in the lessons folder is the longest? answer in one sentence*”. Two calls: one command, then the answer from its result.

```

$ find . -path "*lessons*" -name "quark.py" -exec wc -l {} + 2>/dev/null | sort -n
9 ./lessons/01-model-interface/quark.py
21 ./lessons/02-input-and-output/quark.py
30 ./lessons/03-control-flow/quark.py
48 ./lessons/04-context/quark.py
108 total

The longest is './lessons/04-context/quark.py', at 48 lines.

```

B.5 Context. Three separate runs.

“*what are you, and how do you work? three sentences*”:

I'm quark, an AI agent whose only way of acting and observing is a bash shell. [...] For long-term memory I keep timestamped notes in '.quark/memory/memory.md', so what I learn can carry over between sessions.

“remember that I prefer short answers”: it appended an entry to its memory file and replied:

- User preference: prefers short answers (keep replies brief)

I saved that. I'll keep my answers short.

“what do you know about me?”, in a new session with empty working memory:

```
$ ... cat .quark/memory/memory.md
# Quark Memory

## 2026-10-05 21:56:11
- User preference: prefers short answers (keep replies brief)

Not much yet. My memory has one note about you: you prefer short answers.
```

C The code each production layer adds

Each layer's `quark.py` is the previous one plus the lines below, which are shortened where marked. Where each piece lands is in the comments.

C.1 Observability (+12 lines), in control flow. One function, called at each point the loop already passes: the start, after each model call, after each tool, on each compaction.

```
def trace(**event):                                # observability: append one JSON line per event
    os.makedirs(".quark", exist_ok=True)
    with open(".quark/traces.jsonl", "a") as f: f.write(json.dumps({"ts": datetime.datetime.now().isoformat(
        timespec="seconds"), "run": run, **event}) + "\n")
    ...
trace(event="model", seconds=round(time.time() - start, 2), stop_reason=reply.stop_reason, input_tokens=reply.
    usage.input_tokens, ...)
```

C.2 Guardrails (+26 lines), in control flow. A gate before each tool, and limits before each call.

```
MAX_STEPS, MAX_TOKENS = 20, 200_000
SAFE = {"ls", "cat", "head", "tail", "wc", "grep", "pwd", "date", "echo", "du", "df", "stat", "file", "uniq"}
DENY = re.compile(r"\bsudo\b|rm\s+~\w*[rf]|mkfs|git\s+push|(curl|wget).*\|s*(ba)?sh|\.\env\b")
def guard(cmd):                                     # allow, deny or ask a person
    if DENY.search(cmd): return "blocked by policy"
    if not re.search(r"[;<>`'\n]", cmd) and all((p.split() or [""])[0] in SAFE for p in cmd.split("|")):
        return None
    try: answer = input(f"allow '{cmd}'? [y/N] ").strip()
    except EOFError: answer = ""
    return None if answer.lower() == "y" else f"the person said no: {answer or 'no'}"
    ...
    if steps >= MAX_STEPS or spent >= MAX_TOKENS: ... # top of the loop: stop before another call
    ...
    if (no := guard(block.input["cmd"])):             # the refusal goes back as the tool's result
        results.append({"type": "tool_result", "tool_use_id": block.id, "content": no, "is_error":
            True})
        continue
```

C.3 Sandboxing (+13 lines), in output. A container started once before the loop, and the tool call aimed at it.

```
up = subprocess.run(["docker", "run", "-d", "--rm", "--name", box, "--network", "none", "--memory", "512m", "
    --cpus", "1",
    "--pids-limit", "128", "--cap-drop", "ALL", "--security-opt", "no-new-privileges", "--
    read-only",
```

```

        "--tmpfs", "/tmp", "-e", "HOME=/tmp", "--user", f"{os.getuid()}:{os.getgid()}",
        "-v", f"{where}:{where}", "-w", where, IMAGE, "sleep", "infinity"], ...)
...
done = subprocess.run(["docker", "exec", box, "timeout", "-s", "KILL", str(TIMEOUT), "sh", "-c", block.input[
    "cmd"]], ...)

```

C.4 Resilience (+53 lines), in the model interface, output and context. SDK retries and a backup model; a saved session; interrupted requests answered on resume.

```

client = Anthropic(timeout=300, max_retries=3) # model interface: retries with backoff
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]
def ask(**request): # model interface: backup model, then stop cleanly
    for model in MODELS:
        try: return client.messages.create(model=model, **request)
        except (APIConnectionError, APIStatusError) as e:
            if isinstance(e, APIStatusError) and e.status_code < 500 and e.status_code != 429: raise
            trace(event="model_failed", model=model, error=type(e).__name__)
    raise Down()
def save(task, working_memory): # written atomically after each step
    ...
    os.replace(SESSION + ".tmp", SESSION)
def unfinished(): # context: replay working memory; answer what was lost
    ...
    working_memory.append({"role": "user", "content": [{"type": "tool_result", "tool_use_id": b["id"], "
        content": "interrupted: the harness stopped before this finished, so it may or may not have run. Check
        before repeating it.", "is_error": True} for b in lost]})

```

C.5 Performance (+47 lines), in context, the model interface and output. A cache mark on the conversation, a cap on tool results, streaming, a small model for summaries, and tools run together.

```

MODELS, FAST = ["claude-sonnet-5-5", "claude-opus-5-5"], ["claude-haiku-4-5"] # FAST: compaction only
def ask(models=MODELS, live=False, **request): # model interface: stream the reply
    ...
    with client.messages.stream(model=model, **request) as stream: ...
def trim(text): # context: keep the start and end of a long result
    if len(text) <= MAX_RESULT: return text
    return text[:MAX_RESULT // 2] + f"\n[... {len(text) - MAX_RESULT} characters cut ...]\n" + text[-
        MAX_RESULT // 2:]
def cached(working_memory): # context: mark the end of the conversation for the
    cache
    last = working_memory[-1]
    blocks = [{"type": "text", "text": last["content"]} if isinstance(last["content"], str) else list(last["
        content"])]
    blocks[-1] = {**blocks[-1], "cache_control": {"type": "ephemeral"}}
    return working_memory[-1] + [{"role": last["role"], "content": blocks}]
...
with ThreadPoolExecutor() as pool: # output: run the approved commands at the same time
    outputs = dict(zip(pending, pool.map(execute, pending.values())))

```

C.6 Evaluation (+38 lines), outside the harness. Four cases. Each sets up a fresh folder, runs the real agent on its task, and checks what the agent left behind.

```

def evaluate(names):
    ...
    for case in cases:
        where = tempfile.mkdtemp(prefix=f"eval-{case['name']}")
        subprocess.run(case["setup"], shell=True, cwd=where)
        try: subprocess.run([sys.executable, os.path.abspath(__file__), case["task"]], cwd=where, input="y\n"
            * 50, ..., timeout=300)
        except subprocess.TimeoutExpired: pass
        passed = subprocess.run(case["check"], shell=True, cwd=where, ...).returncode == 0
        ...
    return 1 if failed else 0

```

The baseline, and the same cases after lowering `MAX_STEPS` from 20 to 1:

pass	count	2 steps	4.1s	13435 tokens	
pass	fix	3 steps	5.2s	20262 tokens	
pass	rename	2 steps	3.9s	13428 tokens	
pass	remember	2 steps	4.3s	13582 tokens	
4/4 passed					
pass	count	1 steps	3.0s	6687 tokens	
FAIL	fix	1 steps	2.6s	6644 tokens	kept /tmp/eval-fix-1pf3a9_w REGRESSED: it passed last time
pass	rename	1 steps	2.9s	6674 tokens	
pass	remember	1 steps	3.1s	6778 tokens	
3/4 passed					

D Case-study evidence

D.1 The silent stop and the crash. The response trace that diagnosed it, one line per model call, from the third attempt at the observability lesson. The last response hit the cap partway through a tool request:

```
[trace] stop_reason=tool_use blocks=['thinking', 'tool_use'] out=3642 thinking=2498
[trace] stop_reason=tool_use blocks=['text', 'tool_use'] out=228 thinking=0
[trace] stop_reason=tool_use blocks=['thinking', 'tool_use'] out=148 thinking=70
[trace] stop_reason=max_tokens blocks=['thinking', 'tool_use'] out=4096 thinking=2260
File "lessons/04-context/quark.py", line 37, in <module>
    print(f"$ {block.input['cmd']}")
KeyError: 'cmd'
```

D.2 The agent that killed itself. The command quark ran to test crash recovery while writing the resilience lesson. Its own command line contained 08-resilience/quark.py, so the pattern matched quark too:

```
(yes y | uv run --project $R $R/production/08-resilience/quark.py "Run 'sleep 20; echo first > first.txt' as
one command, ..." > out1.txt 2>&1 &) ; ... ; pkill -9 -f 08-resilience/quark.py; ...
```

D.3 Fabrication from memory. The two decks written from memory were checked slide by slide against the lessons:

- **The four primitive decks:** 22 slides tightened, none invented.
- **The six production decks:** 47 slides corrected and 5 removed.

The removed slides described runs that never happened, quoted cache numbers no run produced, or reversed which model was benched. The artifact keeps quark’s transcripts and its memory file as they were after the runs.

E Where each production layer lives

Layer	Control flow	Input	Context	Model interface	Output
Observability	•				
Guardrails	•		◦ hiding denied tools		
Sandboxing					•
Resilience			◦ replaying saved memory	•	•

Layer	Control flow	Input	Context	Model interface	Output
Performance	◦ routing, in the fuller example		•	•	•
Evaluation	◦ outside the harness, built from control flow, input, model interface and output				

- where the layer lives · ◦ a slice that lands in another primitive